A College Mini Project Report on

ORGAN DONOR REGISTRY MANAGEMENT SYSTEM

Submitted in partial fulfillment of the requirements

for the award of the degree of

BACHELOR OF COMPUTER APPLICATION

Submitted by

[STUDENT NAME]

[Roll / Enrollment No.]

Under the guidance of

[Guide / Faculty Name]

DEPARTMENT OF COMPUTER APPLICATION

[College / Institute Name]

Affiliated to [University Name]

Academic Year 2025 - 2026

Certificate

This is to certify that the project report entitled "Organ Donor Registry Management System" is a bonafide record of the work carried out by [Student Name], bearing Roll / Enrollment Number [Roll No.], in partial fulfillment of the requirements for the award of the degree of Bachelor of Computer Application during the academic year 2025 - 2026.

The work presented in this report is the original work of the student and has been carried out under my supervision and guidance. To the best of my knowledge, this project, or any part of it, has not been submitted elsewhere for the award of any other degree or diploma.

Signature of Guide

[Guide Name]

Signature of Head of Department

[HOD Name]

Date: _____

Signature of External Examiner

Acknowledgement

I would like to express my sincere gratitude to everyone who supported me throughout the development of this project, the Organ Donor Registry Management System.

I am deeply thankful to my project guide, [Guide / Faculty Name], for the valuable guidance, constant encouragement and constructive feedback that shaped this work at every stage. Their suggestions helped me refine both the design and the implementation of the application.

I extend my thanks to the Head of the Department and to all the faculty members of the Department of Computer Application for providing the resources and a supportive learning environment. I am also grateful to my institution, [College / Institute Name], for the opportunity to undertake this project.

Finally, I thank my family and friends for their patience and motivation during the course of this work.

[Student Name]

Abstract

Organ donation saves lives, yet the process of maintaining accurate records of donors and the patients who need organs is often handled manually or with heavyweight software that requires servers and databases. For a small institution or a classroom demonstration, such solutions are impractical.

The Organ Donor Registry Management System is a lightweight, browser-based application that maintains two registries — one for organ donors and one for organ recipients. It allows an operator to register new records, view them in tabular form, edit or delete existing records, and search the donor registry by name, blood group or required organ. A dashboard presents live totals for both registries.

The application is built entirely with HTML5, CSS3 and vanilla JavaScript, styled with a locally bundled copy of Bootstrap 5. All data is stored on the client using the browser's localStorage, which means the system needs no server, no database engine and no internet connection. The complete project runs simply by opening the index.html file in any modern web browser, making it an ideal, self-contained mini project that is easy to distribute, demonstrate and evaluate.

Table of Contents

1. Introduction

The Organ Donor Registry Management System is a simple web application designed to record and manage information about organ donors and organ recipients. Organ transplantation depends on quickly matching a suitable donor with a patient in need, and this matching begins with well-organised records. Even at a small scale, keeping such records on paper or in scattered spreadsheets is error-prone and difficult to search.

This project demonstrates how the core of such a registry can be built using only front-end web technologies. The application presents a clean, professional interface where an operator can register donors and recipients, review the stored records, update or remove them, and search for donors that match specific criteria. Because everything runs inside the web browser and stores data locally, the system is easy to set up and completely free of installation or configuration overhead.

1.1 Overview

The system is organised into five connected pages: a Dashboard, a Donor registration page, a Recipient registration page, a Search page and an About page. A shared navigation bar links these pages together, giving the application the feel of a small single-purpose information system while keeping the codebase minimal and easy to understand.

2. Problem Statement

Hospitals, non-governmental organisations and awareness camps frequently collect the details of people who are willing to donate organs and of patients who are waiting for a transplant. In many small setups this information is written on paper forms or entered into ad-hoc spreadsheets. Such methods create several practical problems:

- Records are easy to lose, duplicate or misplace.
- Searching for a donor with a particular blood group or organ is slow and manual.
- Editing or removing outdated entries is cumbersome and inconsistent.
- Dedicated database-driven software is expensive, needs installation, and requires a server and technical maintenance.

There is therefore a need for a simple, reliable and low-cost tool that can maintain donor and recipient registries, support quick searching, and run on any ordinary computer without special infrastructure. This project addresses that need.

3. Objectives

The main objectives of the Organ Donor Registry Management System are:

- To maintain a structured registry of organ donors with their personal, contact and medical details.
- To maintain a registry of organ recipients along with the organ they require and their hospital.
- To provide complete create, read, update and delete (CRUD) operations for both registries.
- To allow donors to be searched by name, blood group and organ.
- To validate user input so that the stored data remains clean and consistent.
- To present live totals of donors and recipients on a dashboard.
- To deliver a responsive, professional interface that is easy to use.
- To ensure the entire application runs offline, with no server, database or installation.

4. Existing System

In the existing manual approach, donor and recipient information is typically captured on printed forms or in general-purpose spreadsheet files. Locating a specific record means scanning through pages or rows by hand, and there is no built-in validation to prevent incomplete or malformed entries.

Larger organisations may use full-scale hospital management or registry software. While powerful, these systems are built around client-server architectures with a dedicated database engine such as MySQL or PostgreSQL. They must be installed, configured and maintained, they usually require a network connection, and they are far too heavy for a classroom project or a small awareness drive.

4.1 Drawbacks of the Existing System

- Manual searching and sorting is slow and error-prone.
- No automatic validation of the entered data.
- Difficult to update or delete records cleanly.
- Database-driven alternatives are costly and need installation and servers.
- Not suitable for quick, offline, single-machine use.

5. Proposed System

The proposed system is a self-contained web application that replaces the manual registers with an organised, searchable digital registry that still runs on a single machine without any backend. It keeps the two registries — donors and recipients — inside the browser's localStorage and provides an intuitive interface for managing them.

5.1 Key Features

- Dashboard showing live counts of total donors and total recipients.
- Donor registration with auto-generated IDs and full CRUD support.
- Recipient registration with auto-generated IDs and full CRUD support.
- Search of the donor registry by name, blood group and organ.
- Field validation with friendly, on-screen messages.
- Data persistence through localStorage, surviving page reloads.
- A clean, responsive, blue-themed user interface.

5.2 Advantages over the Existing System

Compared with manual registers, the proposed system offers instant searching, automatic ID generation, input validation and effortless editing. Compared with heavyweight database software, it needs no installation, no server and no internet connection — it runs the moment the index.html file is opened.

6. Scope

The scope of the project defines what the system does and the boundaries within which it operates.

6.1 In Scope

- Registration, listing, editing and deletion of organ donors.
- Registration, listing, editing and deletion of organ recipients.
- Searching donors by name, blood group and organ.
- Displaying dashboard totals for donors and recipients.
- Client-side validation of all form inputs.
- Storing all data locally in the browser via localStorage.

6.2 Out of Scope

- User authentication, roles and access control.
- Automatic donor-to-recipient matching or allocation.
- Central or networked storage shared between multiple computers.
- Integration with hospital systems or government databases.

7. Functional Requirements

The functional requirements describe what the system must do:

1. The system shall allow the operator to register a donor with an auto-generated ID and the fields: full name, age, gender, blood group, phone number, address, organ to donate and an optional medical condition.
2. The system shall allow the operator to register a recipient with an auto-generated ID and the fields: full name, age, gender, blood group, required organ and hospital name.
3. The system shall display all registered donors and recipients in tabular form.
4. The system shall allow the operator to edit any existing donor or recipient record.
5. The system shall allow the operator to delete any donor or recipient record after confirmation.
6. The system shall allow the operator to search donors by name, blood group and organ, individually or in combination.
7. The system shall show the total number of donors and recipients on the dashboard.
8. The system shall validate all required fields and reject invalid data with a clear message.
9. The system shall store and retrieve all records using the browser’s localStorage.

8. Non Functional Requirements

The non-functional requirements describe how the system should perform:

- Usability: The interface must be clean, consistent and easy to learn, with clear labels and messages.
- Portability: The application must run in any modern web browser on any operating system.
- Availability: The system must work fully offline, without any server or internet connection.
- Performance: Pages and searches must respond instantly for the expected small to moderate data volumes.
- Reliability: Saved data must persist across page reloads and browser restarts on the same device.
- Maintainability: The JavaScript code must be modular, commented and free of unnecessary duplication.
- Responsiveness: The layout must adapt to different screen sizes, from mobile to desktop.

9. Feasibility Study

A feasibility study evaluates whether the project is practical to build and use.

9.1 Technical Feasibility

The system uses only standard, widely-supported web technologies — HTML5, CSS3, JavaScript, Bootstrap 5 and the localStorage API. These are available in every modern browser and require no special hardware or software. The project is therefore technically very feasible.

9.2 Economic Feasibility

The application relies entirely on free and open technologies. There are no licensing costs, no server hosting charges and no database expenses. As a result, the project is economically feasible and essentially free to build and run.

9.3 Operational Feasibility

The interface is simple enough that any operator can use it after a brief introduction. Because the system runs by opening a single file, deployment is trivial and day-to-day operation needs no technical expertise, making it operationally feasible.

10. System Architecture

The application follows a simple, layered client-side architecture. All processing happens inside the user's browser. The presentation layer (HTML pages and Bootstrap styling) captures user actions and displays results. The logic layer (the page scripts) validates input and coordinates the workflow. A dedicated data-access layer (storage.js) is the only component that reads from and writes to localStorage, which acts as the persistent store.

Figure 10.1: System architecture of the Organ Donor Registry Management System

This separation keeps the storage details in one place, so the rest of the code works through a small set of well-named functions. It also makes the system easy to understand, test and extend.

11. Module Description

The system is divided into the following modules:

11.1 Dashboard Module

Implemented in index.html, this module displays the total number of donors and recipients and provides navigation cards to the registration pages.

11.2 Donor Module

Implemented in donors.html and donor.js, this module handles donor registration, validation, listing, editing and deletion.

11.3 Recipient Module

Implemented in recipients.html and recipient.js, this module handles recipient registration, validation, listing, editing and deletion.

11.4 Search Module

Implemented in search.html and search.js, this module filters the donor registry by name, blood group and organ and displays the matching donors.

11.5 Storage Module

Implemented in storage.js, this module is the data-access layer. It reads and writes the donor and recipient collections in localStorage and generates unique IDs.

11.6 Common Module

Implemented in common.js, this module provides shared utilities: the navigation bar, alert messages, input-validation helpers and drop-down population.

11.7 About Module

Implemented in about.html, this module presents the project description, objectives and technologies used.

12. Data Design (localStorage-based)

The application does not use a traditional database. Instead, it stores data as JSON text in the browser's localStorage under two keys. Two additional keys hold the running counters used to generate unique IDs.

localStorage Key	Purpose
odr_donors	JSON array of donor records
odr_recipients	JSON array of recipient records
odr_donor_seq	Counter for generating donor IDs
odr_recipient_seq	Counter for generating recipient IDs

12.1 Donor Record Structure

Field	Type	Description
id	String	Auto-generated donor ID, e.g. DNR-0001
name	String	Full name of the donor
age	Number	Age in years
gender	String	Male / Female / Other
bloodGroup	String	Blood group, e.g. O+
phone	String	Contact number (digits only)
address	String	Residential address
organ	String	Organ the person wishes to donate
medicalCondition	String	Optional medical notes

12.2 Recipient Record Structure

Field	Type	Description
id	String	Auto-generated recipient ID, e.g. RCP-0001
name	String	Full name of the recipient
age	Number	Age in years
gender	String	Male / Female / Other
bloodGroup	String	Blood group, e.g. A+
organ	String	Organ required by the patient
hospital	String	Hospital where the patient is treated

13. ER Diagram

Although the system stores data in localStorage rather than in a relational database, the information can still be modelled as two entities — Donor and Recipient. The entity-relationship diagram below shows their attributes. A donor and a recipient are conceptually related when their blood group and organ are compatible.

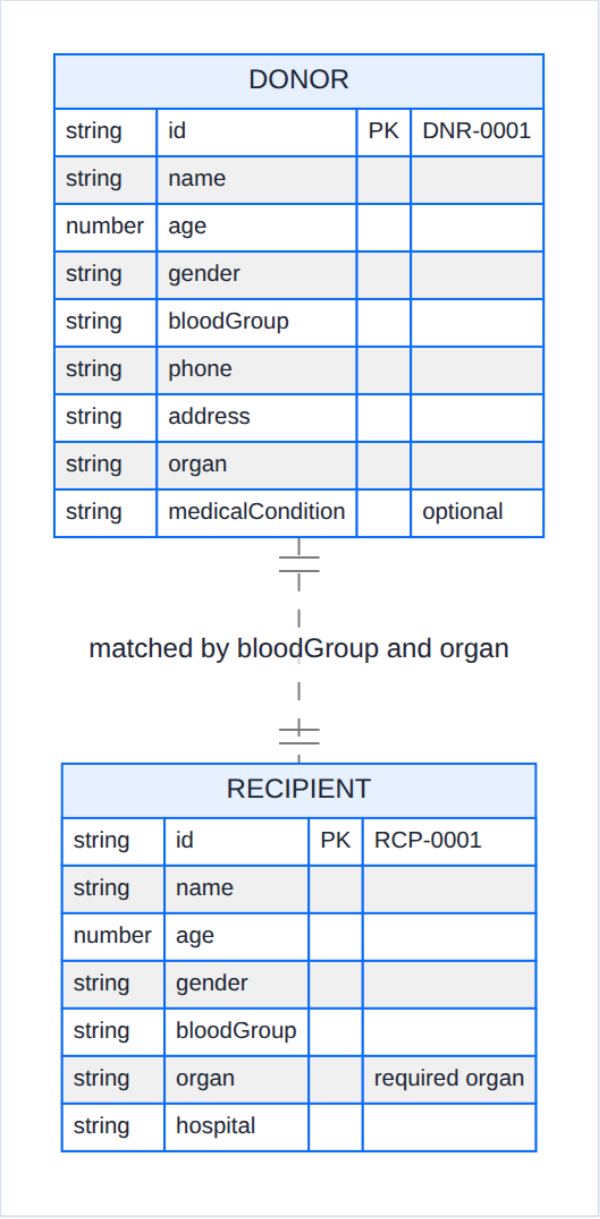

Figure 13.1: Entity-Relationship diagram of the registry

14. Data Flow Diagram

The data flow diagrams describe how information moves through the system. The context-level (Level 0) diagram treats the whole application as a single process interacting with the operator and the localStorage store.

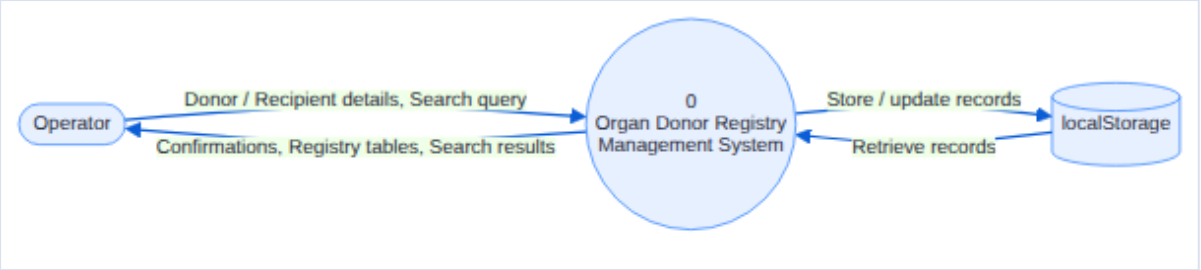

Figure 14.1: Level 0 (context) Data Flow Diagram

The Level 1 diagram expands the single process into the main functions of the system: managing donors, managing recipients, searching donors and showing dashboard totals.

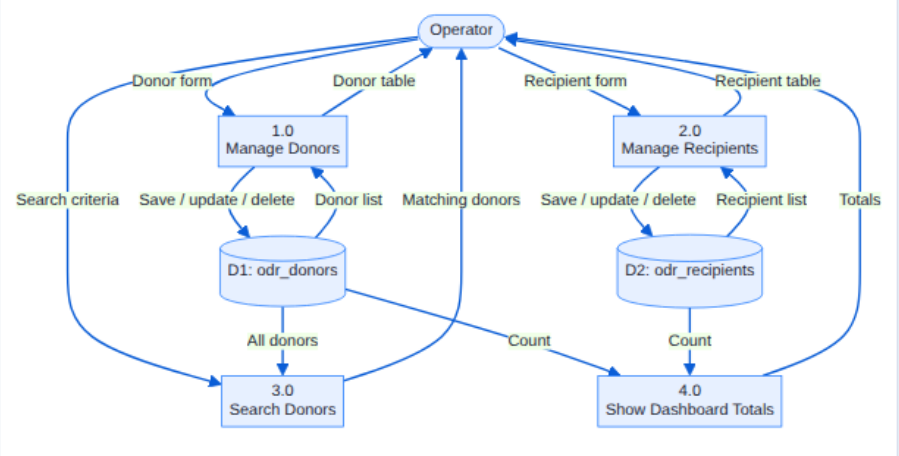

Figure 14.2: Level 1 Data Flow Diagram

15. Flowcharts

The following flowcharts illustrate the two most important workflows in the system: adding or updating a record, and searching for donors.

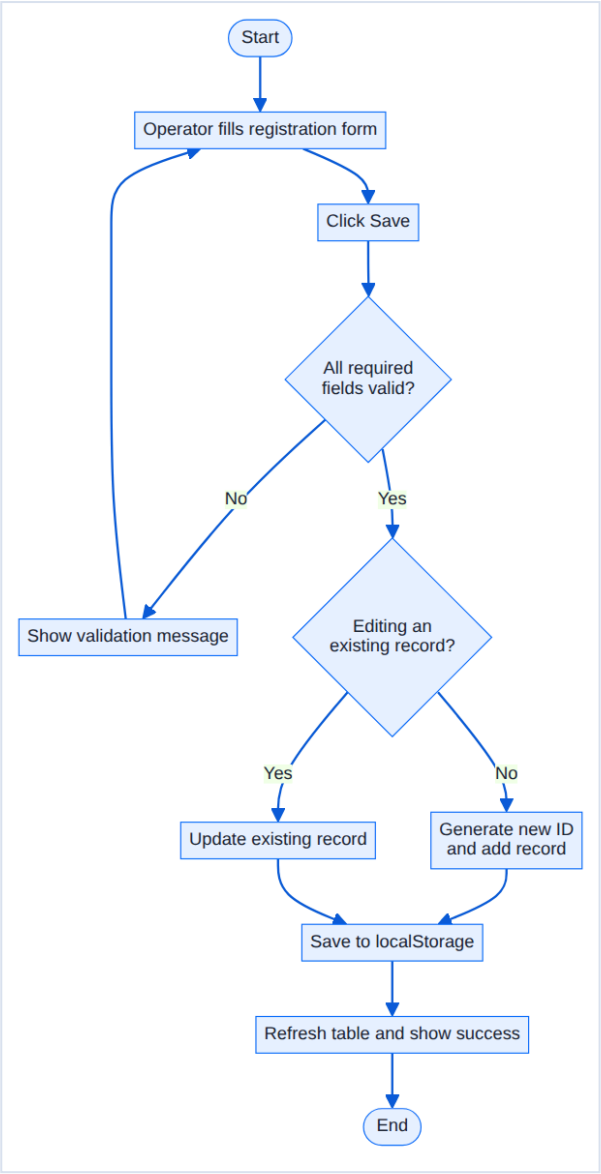

Figure 15.1: Flowchart for adding or updating a record

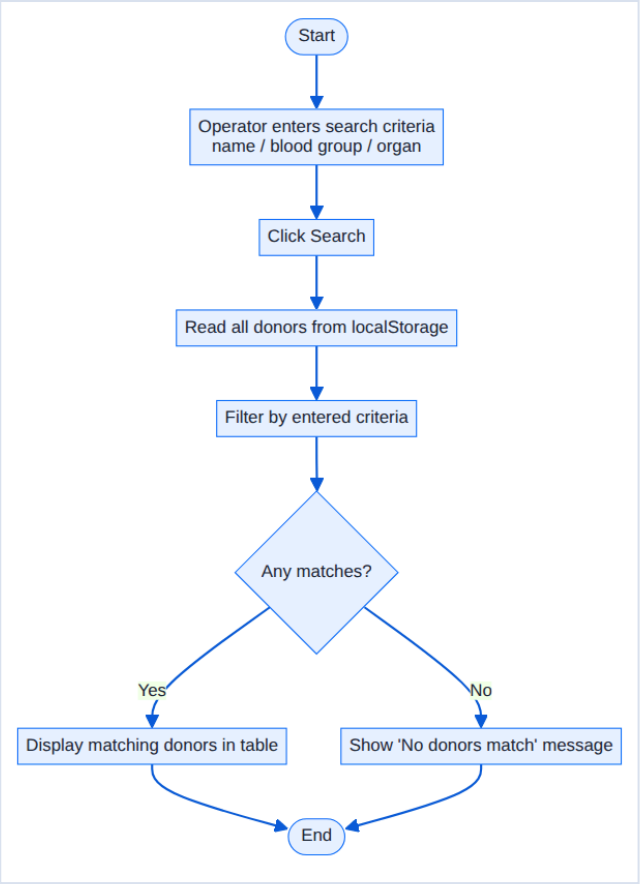

Figure 15.2: Flowchart for searching donors

16. UI Screenshots

The screenshots below are captured from the completed, working application and show each page populated with sample data.

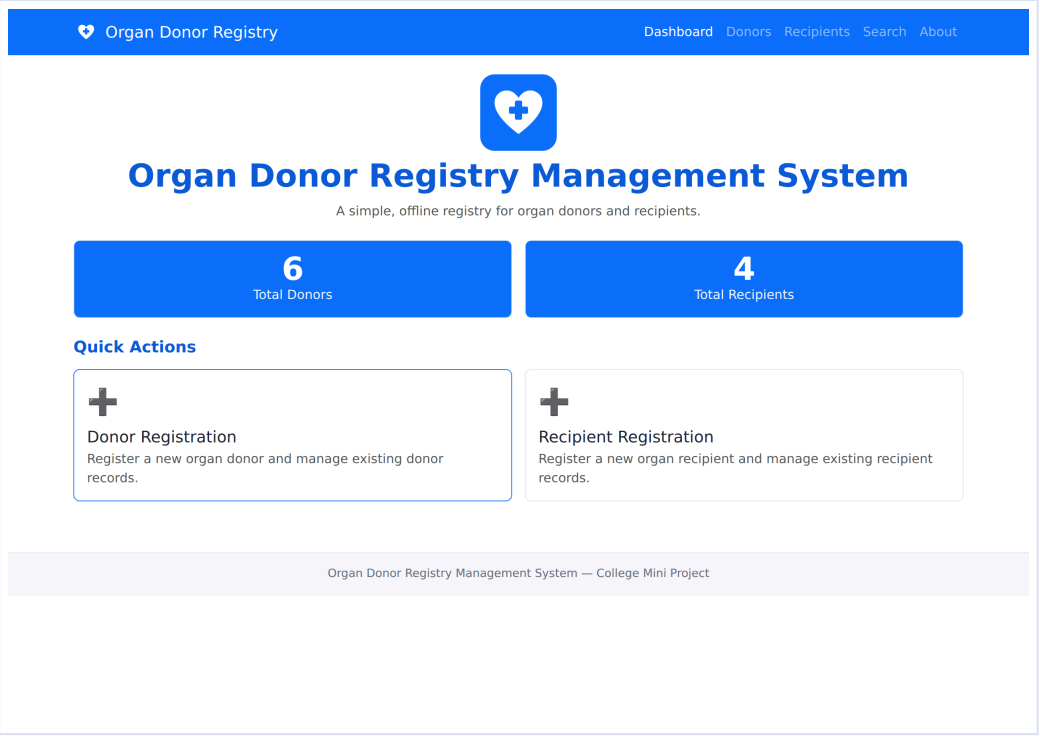

Figure 16.1: Dashboard with live donor and recipient totals

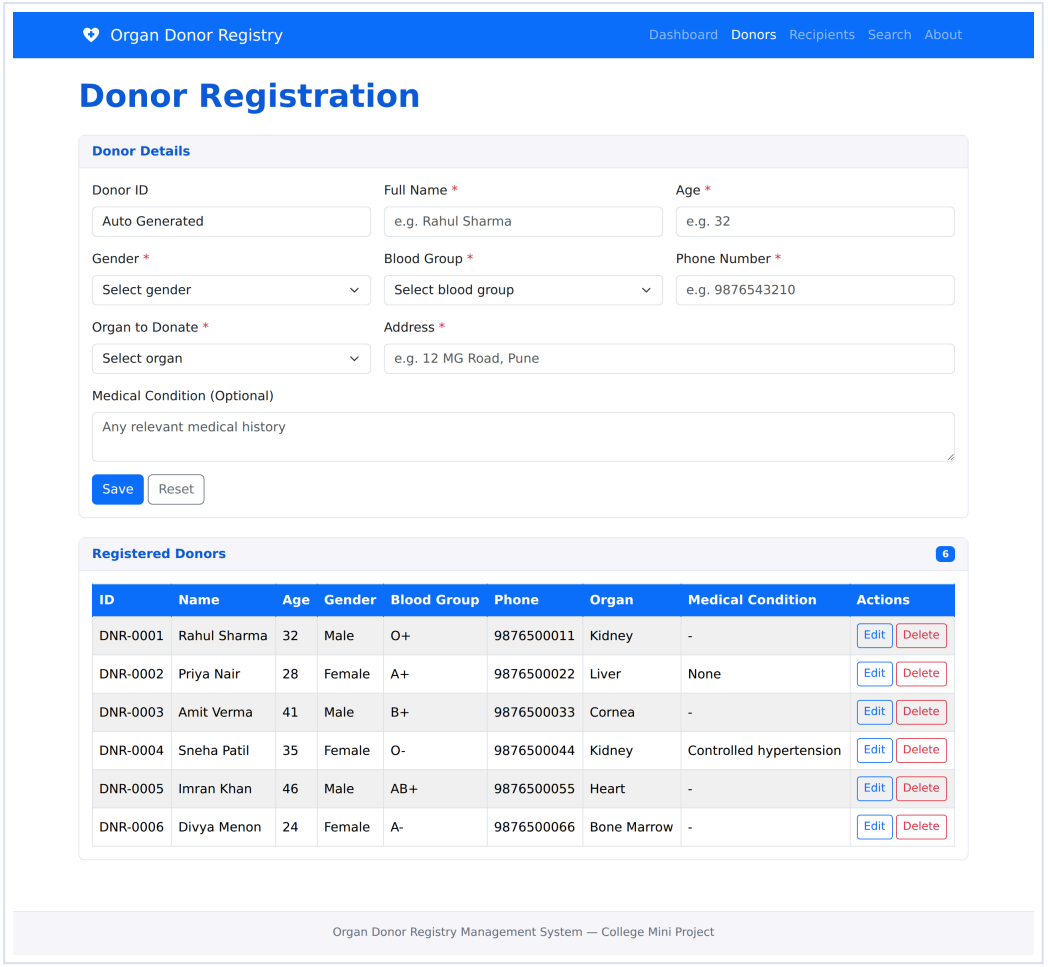

Figure 16.2: Donor registration page with the list of registered donors

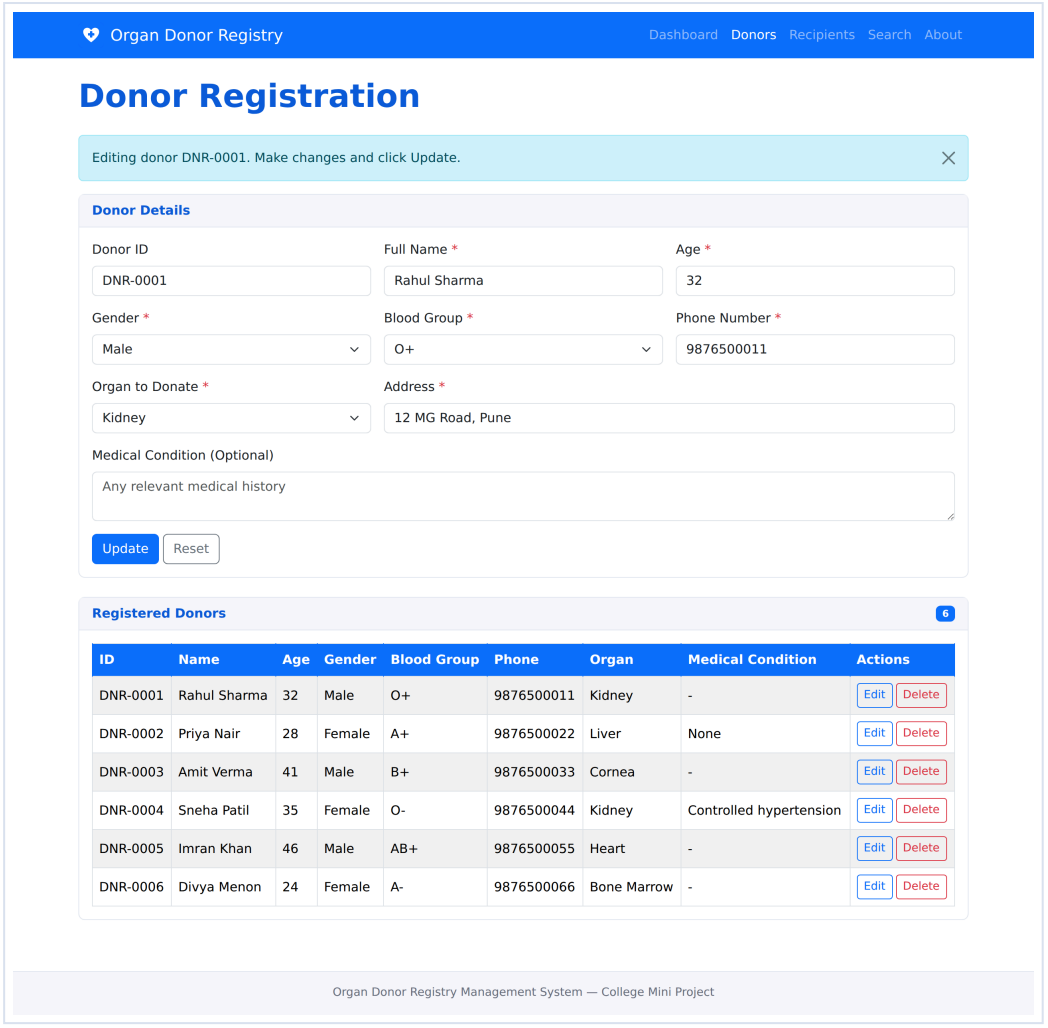

Figure 16.3: Editing an existing donor record

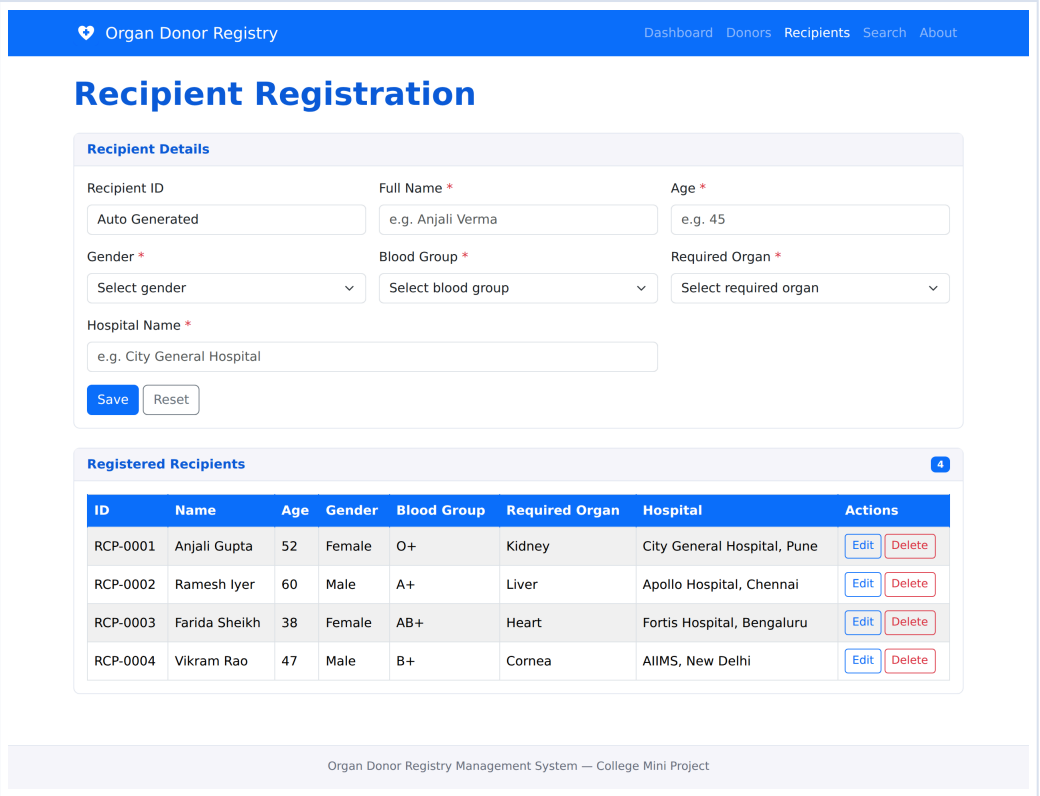

Figure 16.4: Recipient registration page with the list of registered recipients

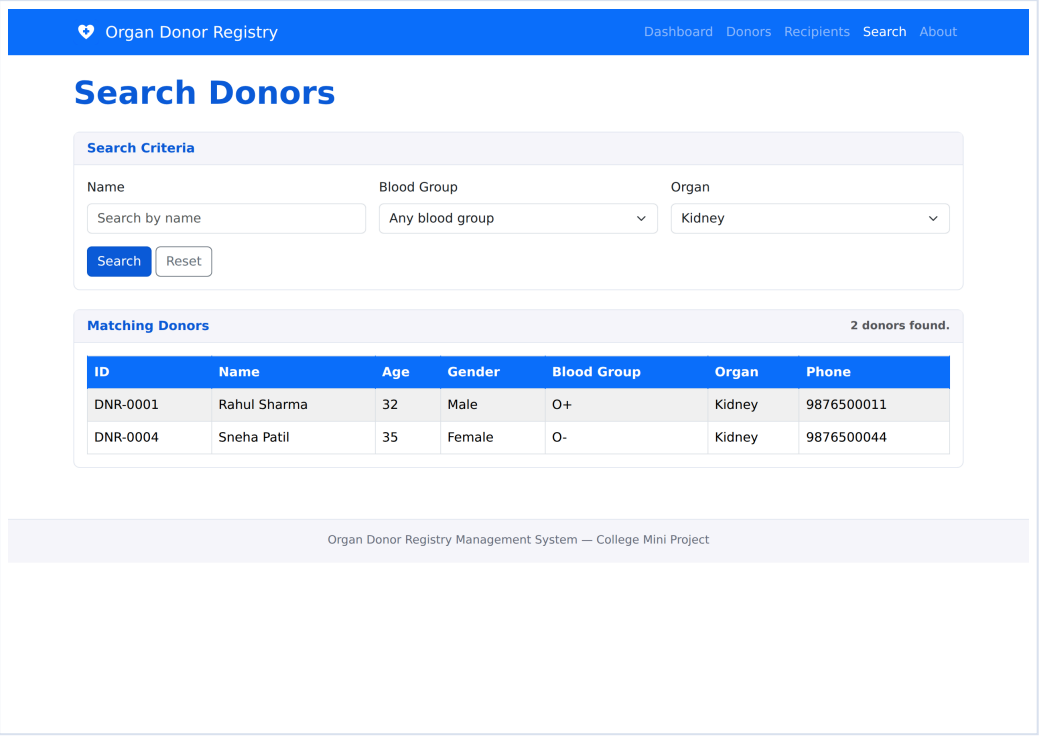

Figure 16.5: Search page filtering donors by organ

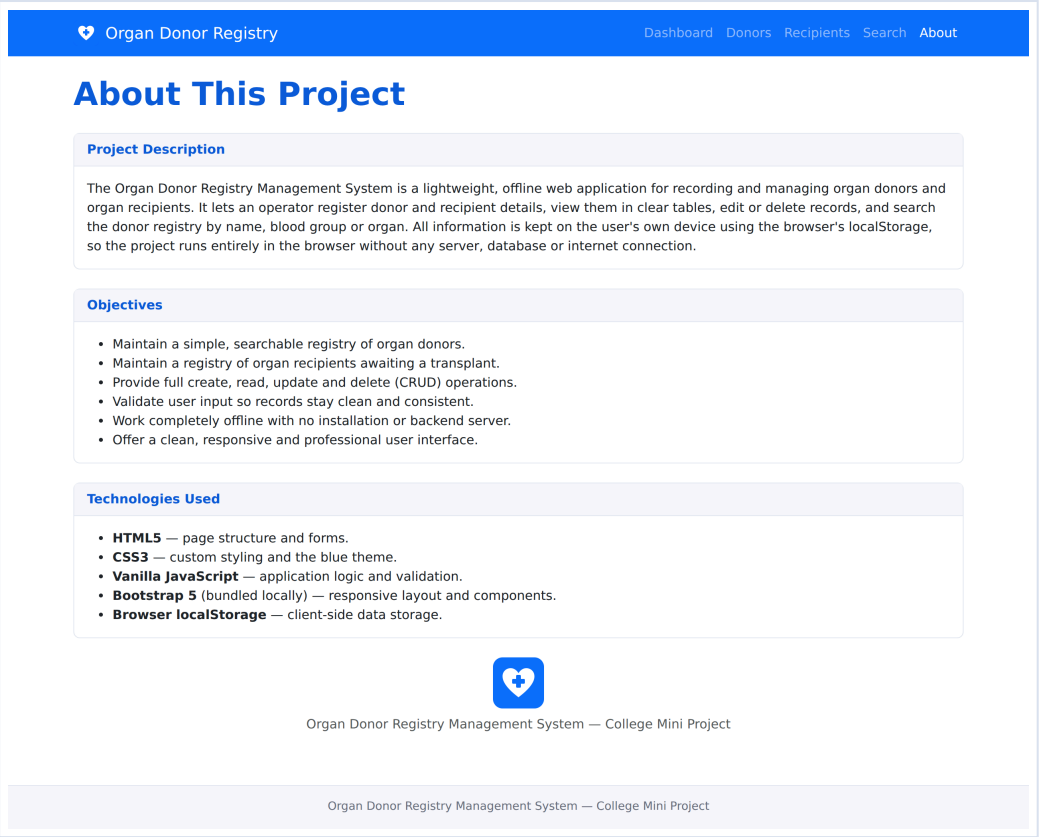

Figure 16.6: About page describing the project

17. Testing

The application was tested by exercising each feature directly in the browser. The table below summarises the main test cases and their results. All listed cases passed.

#	Test Case	Expected Result	Status
1	Open index.html	Dashboard loads and shows totals	Pass
2	Submit empty donor form	Validation message is shown	Pass
3	Enter non-numeric age	Age validation error is shown	Pass
4	Enter non-digit phone number	Phone validation error is shown	Pass
5	Save a valid donor	Donor is added with ID DNR-0001	Pass
6	Edit a donor	Record is updated, no duplicate row	Pass
7	Delete a donor	Record is removed after confirmation	Pass
8	Save a valid recipient	Recipient is added with ID RCP-0001	Pass
9	Search donors by name	Only matching donors are listed	Pass
10	Search donors by organ	Only donors of that organ are listed	Pass
11	Search donors by blood group	Only matching donors are listed	Pass
12	Search with no match	A "no donors match" message is shown	Pass
13	Reload the page	Previously saved data is still present	Pass
14	Navigate between pages	All navigation links work correctly	Pass

18. Advantages

- Runs instantly by opening a single HTML file — no installation.
- Works completely offline, with no server or internet required.
- No database engine to install, configure or maintain.
- Clean, responsive and professional user interface.
- Automatic ID generation and input validation keep data consistent.
- Fast searching of the donor registry by multiple criteria.
- Data persists across page reloads on the same device.
- Modular, well-commented code that is easy to understand and extend.
- Free to build and distribute; easy to zip and submit.

19. Limitations

- Data is stored only in one browser on one device and is not shared across machines.
- Clearing the browser data or storage removes all saved records.
- There is no user authentication or access control.
- The system does not perform automatic donor-to-recipient matching.
- localStorage has a limited capacity, so the system suits small to moderate data volumes.
- No automatic backup; records are not exported to an external file.

20. Future Scope

The project can be extended in several directions in the future:

- Add user authentication and role-based access for operators and administrators.
- Introduce automatic matching of recipients to compatible donors by blood group and organ.
- Provide import and export of records to and from CSV or JSON files for backup.
- Connect to a central database and server so multiple centres can share one registry.
- Add reports and statistics, such as organ-wise and blood-group-wise summaries.
- Send notifications or reminders when a suitable match becomes available.

21. Conclusion

The Organ Donor Registry Management System successfully demonstrates how a useful, real-world registry can be built using only front-end web technologies. It provides complete management of donor and recipient records, quick searching, input validation and a clear dashboard, all within a clean and responsive interface.

By storing data in the browser's localStorage, the system avoids the cost and complexity of servers and databases while still delivering reliable, persistent storage on the user's device. The result is a self-contained application that runs the moment its index.html file is opened, making it an ideal college mini project — easy to build, easy to demonstrate and easy to evaluate. The project also lays a solid foundation that can be expanded into a fuller system as described in the future scope.

22. References

1. MDN Web Docs — HTML: HyperText Markup Language. Mozilla Developer Network.
2. MDN Web Docs — CSS: Cascading Style Sheets. Mozilla Developer Network.
3. MDN Web Docs — JavaScript Guide. Mozilla Developer Network.
4. MDN Web Docs — Window.localStorage and the Web Storage API. Mozilla Developer Network.
5. Bootstrap 5 Documentation — Layout, Components and Forms. The Bootstrap Team.
6. World Wide Web Consortium (W3C) — HTML Living Standard.
7. ECMAScript Language Specification (ECMA-262).